

Hands On 1

Angelo Savino

November 17, 2024

Exercise 1: Min and Max

The first exercise requires us to support the following queries over an array a :

- $\text{Update}(i, j, T)$: update all elements $a[k]$ for $k \in [i, j]$ with $\min(a[k], T)$.
- $\text{Max}(i, j)$: return the maximum value $a[k]$ for $k \in [i, j]$

Given that we need to support range update queries, and we need to support them in $O(\log(n))$ time, we need to solve this problem using a segment tree with lazy evaluation.

The tree is built in a recursive fashion, and each node representing a given range $[i, j]$ (with $i \leq j$), contains the maximum value in the range, being the answer for the $\text{Max}(i, j)$ query. The value for an internal node is simply the maximum value of the node's children. The tree is represented in an implicit way using an array, where the index 0 represents the root of the segment tree and the left and right children of a node i are found respectively at position $2i + 1$ and $2i + 2$. At build time is also created an auxiliary array *lazy* that will store the pending updates for each node.

When an update query is issued, we stop at the first nodes that result in a total overlap, and we update the solution for the node, then we store the update in the respective position in *lazy*. In this way we avoid performing unnecessary updates in the relative subtree, performing them at a later time only if needed, i.e. if we need to traverse the given node during an *Update* or *Max* query. The update function for a given node first takes the minimum value between the node's solution s and T , because if T is greater than s would not be substituted in the range, because all values in the range are less than s , while if $T < s$ all the values bigger than T in the range would be replaced with T , making it the biggest element in the range after the update.

The Max query is performed normally by combining all solutions in the nodes representing full overlaps using the `max` function and returning the value to the caller.

The construction of the segment tree requires linear time, by first filling the leaf nodes with the values of the original array, then filling each internal node in $O(1)$. Both the *Update* and *Max* queries run in $O(\log n)$ time each. The total running time of the solution is $O(n + (m \log n))$.

Exercise 2: Is There

In this exercise we are given n segments $[l, r]$, and we are required to support the *IsThere*(i, j, k) query, where we return 1 if there exists a point $p \in [i, j]$ that is contained by exactly k segments, 0 otherwise. This can be done by first counting the number of segments present in each point using a sweep line algorithm, then constructing a segment tree over the resulting array. A node in this segment tree will contain an *HashSet* that with all the distinct values appearing in the interval represented by the node. Each internal node is obtained by merging its children's *HashSets*.

An *IsThere*(i, j, k) query is resolved by performing an `HashSet::contains` query in each of the nodes that result in a total overlap, then returning 1 if at least one of those return true.

The construction of the array using sweep line runs in $O(n)$ time using prefix sums, the construction of the segment tree requires at most n *HashSet* insertions per level, which require $O(1)$ time each. Each *IsThere* query runs in $O(\log n)$ time. The total running time of the solution is thus $O((n + m) \log n)$.